Module 9: Counterfactual Explanations

Instructional Hours: 6

Module Overview

In this module you will learn about counterfactual explanations. This is technique that enables you to see how you can modify the feature values of an instance so that its corresponding prediction can change to some desirable value. You will also get the implement counterfactual explanations in python code.

Learning Outcomes

At the end of this module, you will be able to:

1. Explain the principles behind use of Counterfactual Explanations in Making Al models interpretable
2. Describe The Process of Generation of Counterfactual Explanations for an Al model
3. Create code to implement Counterfactual Explanations in Python
4. Interpret Counterfactual Explanations to stakeholders

Learning Activities/task List

1. Review Introductory notes on of Counterfactual Explanations
2. Contribute to a discussion on of Counterfactual Explanations
3. Review selected readings and examples on of Counterfactual Explanations
4. Complete a Quiz on of Counterfactual Explanations
5. Implement Python code to generate of Counterfactual Explanations
6. Complete a Final Module Quiz

Introduction to Counterfactual Explanations

A counterfactual explanation provides an alternative scenario by altering input features of a machine learning model to produce a different outcome. It answers what if questions. They help understand model decisions by highlighting what changes in the input would lead to a different (often desirable) prediction.

The original scenario is the model's actual prediction based on input data. The counterfactual scenario alters certain features to observe how the model's output changes.

As an example, a loan application is rejected based on the applicant's current salary and credit score. A counterfactual explanation might show that if the salary were 10% higher, the loan would be approved.

Counter Factual explanations work by modifying one or more features of the input data while keeping the rest constant. The counterfactual should be as close as possible to the original instance (minimal change) to ensure realistic and actionable explanations.

Generating Counterfactual Explanations

The following is the sequence of steps in generating counterfactual explanations:

1. Start with the original instance and observe the model's prediction.
2. Decide what different outcome you want the model to predict (e.g., changing a "no" decision to a "yes").
3. Adjust the features of the instance to achieve the desired output.
4. Check if the modification results in the desired prediction.

Approaches to Generating Counterfactual Explanations

1. **Brute Force Search**
   This works by testing different combinations of feature changes to see which ones lead to the desired outcome.
2. **Gradient-based Methods**
   This uses gradient information to adjust the input features towards the desired output. It works well with differentiable models.
3. **Evolutionary Algorithms**
   Using genetic algorithms to search for feature combinations that result in the desired prediction.
4. **Optimization-based Methods**
   Formulating the counterfactual search as an optimization problem where the objective is to find the minimal change in input features that results in a different prediction.

Examples of Counterfactual Explanations

In credit scoring, if a model denies a person a loan based on a credit score of 650. A counterfactual explanation would be that the model would approve the loan if the credit score were 700 , and/or if the annual income were increased by 10%.

In medical diagnosis if model predicts that a patient has a high risk of developing diabetes. An example counterfactual explanation would be that if the patient's BMI were lower by 5 units, the model would predict a lower risk.

In a hiring situation if a model rejects an applicant for a job, an example counter factual explanation would be if the applicant had an additional 2 years of experience, they would be considered for the position.

Desirable Properties of Counterfactual Explanations

1. Sparsity is the requirement that only a small number of features should be modified. This ensures the explanation is simple and actionable.
2. Realism is the requirement that the modified features should be plausible and realistic. Counterfactuals that suggest impossible changes (e.g., a person growing taller) are not useful.
3. Actionability is the counterfactual should offer actionable insights (e.g., increasing salary, taking a course to improve skills).
4. Diversity is the requirement that multiple counterfactual explanations for the same prediction can provide different actionable paths.
5. Causal Validity is the requirement that changes should respect causal relationships between features. For instance, increasing a person's salary should not decrease their age in the data.

Challenges and Limitations Counterfactual Explanations

1. **Computational Cost** - Finding the minimal change that leads to a different outcome can be computationally expensive, especially for large models.
2. **Multiple Solutions** - Many counterfactuals may exist for the same instance, leading to ambiguity.
3. **Unrealistic Recommendations** - Some counterfactuals may suggest unrealistic changes (e.g., changing immutable features like age).
4. **Non-Causal Models** - Counterfactual explanations are most effective when applied to causal models. Without causality, the explanation may not reflect a true path to changing the outcome.

Detailed Review of Counterfactual Explanations

Reading Material

To learn more about Counterfactual Explanations, read the following materials Counterfactual Explanations

- Molnar, C. (2022).Interpretable Machine Learning:A Guide for Making Black Box Models Explainable (2nd ed.).

Implementation of Anchors Using Python

You can implement the Counterfactual explanations using python.

```
#!pip install alibi[tensorflow]
#import the necessary Libraries
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from alibi.explainers import CounterfactualRL, Counterfactual
from matplotlib import pyplot as plt
import tensorflow as tf
from sklearn.datasets import load_iris
import pandas as pd
import numpy as np


#load the dataset
data=load_iris()
df= pd.DataFrame(data.data,columns=data.feature_names)
columns=data.feature_names


df['target']=data.target


df.head()


y=data.target
x=data.data


# Split the data into training and testing sets
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)
```

```
# Standardize the features
scaler = StandardScaler()
x_train_scaled = scaler.fit_transform(x_train)
x_test_scaled = scaler.transform(x_test)


#the function for creating an outoencoder
from tensorflow.keras.layers import Dense
from tensorflow import keras


ENCODING_DIM = 7
BATCH_SIZE = 64
EPOCHS = 10


class AE(keras.Model):
    def __init__(self, encoder: keras.Model, decoder: keras.Model, **kwargs) -> None:
        super().__init__(**kwargs)
        self.encoder = encoder
        self.decoder = decoder


    def call(self, x: tf.Tensor, **kwargs):
        z = self.encoder(x)
        x_hat = self.decoder(z)
        return x_hat


def make_ae():
    len_input_output = x_train_scaled.shape[-1]


    encoder = keras.Sequential()
    encoder.add(Dense(units=ENCODING_DIM*2, activation="relu", input_shape=(len_input_output
    encoder.add(Dense(units=ENCODING_DIM, activation="relu"))


    decoder = keras.Sequential()
    decoder.add(Dense(units=ENCODING_DIM*2, activation="relu", input_shape=(ENCODING_DIM, ))
    decoder.add(Dense(units=len_input_output, activation="linear"))


    ae = AE(encoder=encoder, decoder=decoder)
```

```
    ae.compile(optimizer='adam', loss='mean_squared_error')
    history = ae.fit(x_train_scaled, x_train_scaled,
        batch_size=BATCH_SIZE,
        epochs=EPOCHS,
        verbose=True,)
    # loss = history.history['loss']
    # plt.plot(loss)
    # plt.xlabel('Epoch')
    # plt.ylabel('MSE-Loss')
    return ae

#Create the autoencoder
ae=make_ae()

#A function to show the differences btn any two instances
def compare_instances(x, cf):
    """
    Show the difference in values between two instances.
    """
    x = x.astype('float64')
    cf = cf.astype('float64')
    for f, v1, v2 in zip(features, x[0], cf[0]):
        print(f'{f:<25} instance: {round(v1, 3):^10} counter factual: {round(v2, 3):^10} dif


# Create and train a Random   Forest model
model = RandomForestClassifier(random_state=42)
model.fit(x_train_scaled, y_train)


#create a prediction function
predict_fn = lambda x: model.predict_proba(x.reshape(x.shape[0],-1))


pred=predict_fn(x_test_scaled)[0:5]
print(pred)


  # A function to reshape an instance to the format required by the model
def reshape(x):
  return x.reshape(x.shape[0],-1)


cfrl_explainer = CounterfactualRL(
```

```
    predictor=predict_fn,  # The model to explain
    encoder=ae.encoder,  # The encoder
    decoder=ae.decoder,  # The decoder
    latent_dim=7,  # The dimension of the autoencoder latent space
    coeff_sparsity=0.5,  # The coefficient of sparsity
    coeff_consistency=0.5,  # The coefficient of consistency
    train_steps=10,  # The number of training steps
    # Change batch_size to a smaller value to avoid empty batches during fit
    # Using a smaller batch size for the explainer, preferably a factor of the training data
    batch_size=10,  # Changed from 24
)

# Remove the reshape function call:
cfrl_explainer.fit(X=x_train_scaled) # Pass the data directly


#Explain an instance
x=x_test_scaled[0].reshape(1,-1)
result_cfrl = cfrl_explainer.explain(X=x, Y_t=np.array([1]))
print("Instance class prediction:", data.target_names[model.predict(x)])
countFactPred=model.predict(result_cfrl.data['cf']['X'])[0].argmax()
print("Counterfactual class prediction:", data.target_names[countFactPred])
x=scaler.inverse_transform(x)
counterFact=scaler.inverse_transform(result_cfrl.data['cf']['X'])


features=columns
compare_instances(x, counterFact)
```